

Ejercicios en clase

Carlos Santiago Martinez Torres

2025-05-25

Contents

Configuracion inicial	1
Carpetas	1
Ejercicio Tema 4. tidy iris.	2
Carga del conjunto	2
Numero de registros y tipos de variables	2
Variables perdidas. NA.	2
Transformaciones	2
Ejercicio Tema 4. Baby boom.	3
Lectura de fichero	3
Numero de registros y tipos de variables	3
Variables perdidas. NA.	3
Transformaciones	3
Representacion	4
Ejercicio Tema 6. Textos.	6
Funciones basicas para manejo de caracteres en R	7
Otro ejercicio	9

Configuracion inicial

Carpetas

```
# Creamos directorio data si no existe, para guardar los ficheros
if (!file.exists('data')) { dir.create('data') }

# Creamos directorio figure si no existe, para guardar las figuras
```

```
if (!file.exists('figure')) { dir.create('figure') }

# Creamos los accesos a cada carpeta
datos <- 'data/'
figuras <- 'figure/'
```

Ejercicio Tema 4. tidy iris.

Carga del conjunto

```
# Ya está en R. Lo trabajamos con otra variable
dfiris <- iris
```

Numero de registros y tipos de variables

```
str(dfiris)

## 'data.frame':   150 obs. of  5 variables:
## $ Sepal.Length: num  5.1 4.9 4.7 4.6 5 5.4 4.6 5 4.4 4.9 ...
## $ Sepal.Width : num  3.5 3 3.2 3.1 3.6 3.9 3.4 3.4 2.9 3.1 ...
## $ Petal.Length: num  1.4 1.4 1.3 1.5 1.4 1.7 1.4 1.5 1.4 1.5 ...
## $ Petal.Width : num  0.2 0.2 0.2 0.2 0.2 0.4 0.3 0.2 0.2 0.1 ...
## $ Species      : Factor w/ 3 levels "setosa","versicolor",...: 1 1 1 1 1 1 1 1 1 1 ...

# glimpse(df)
# dim(df)
```

Variables perdidas. NA.

```
sum(is.na(dfiris))
```

```
## [1] 0
```

Transformaciones

```
# Pasar a conjunto tidy
# dfiris <- pivot_longer(data = dfiris, names_to = 'Parte', values_to = 'Valor', -Species)
dfiris <- dfiris %>%
  pivot_longer(data = ., names_to = 'Parte', values_to = 'Valor', -Species)

# Separar las columnas y organizar
# ?separate()
dfiris <- dfiris %>%
```

```
separate(data = ., col = Parte, into = c('ParteFlor', 'TipoMedida'), sep = '[.]') %>%
select(ParteFlor, TipoMedida, Valor, Species)

# Verificamos el conjunto
head(dfiris)
```

```
## # A tibble: 6 x 4
##   ParteFlor TipoMedida Valor Species
##   <chr>      <chr>      <dbl> <fct>
## 1 Sepal     Length      5.1 setosa
## 2 Sepal     Width       3.5 setosa
## 3 Petal     Length      1.4 setosa
## 4 Petal     Width       0.2 setosa
## 5 Sepal     Length      4.9 setosa
## 6 Sepal     Width       3   setosa
```

Ejercicio Tema 4. Baby boom.

Lectura de fichero

```
f <- paste0(datos, 'children_per_woman_total_fertility.csv')
df <- read_csv(file = f, show_col_types = FALSE)
```

Numero de registros y tipos de variables

```
# str(df)
# glimpse(df)
dim(df)
```

```
## [1] 197 302
```

Variables perdidas. NA.

```
sum(is.na(df))
```

```
## [1] 0
```

Transformaciones

```
# Convertir a un conjunto tidy
df <- pivot_longer(data = df, names_to = 'Year', values_to = 'Value', cols = -country) # Hacerlo una ve
```

```

# Verificacion de valores
str(df) # El año no es necesario cambiarlo porque no operaremos aritmeticamente con el

## tibble [59,297 x 3] (S3: tbl_df/tbl/data.frame)
## $ country: chr [1:59297] "Afghanistan" "Afghanistan" "Afghanistan" "Afghanistan" ...
## $ Year : chr [1:59297] "1800" "1801" "1802" "1803" ...
## $ Value : num [1:59297] 7 7 7 7 7 7 7 7 7 ...

# Actualizamos para trabajar mejor los filtros
df <- df %>%
  mutate(Year = as.integer(Year)) %>% # Cambiamos el tipo de dato
  rename(Country = country) %>% # Cambiamos 'country' a 'Country'
  select(Country, Year, Value) # Seleccionamos en orden para organizar

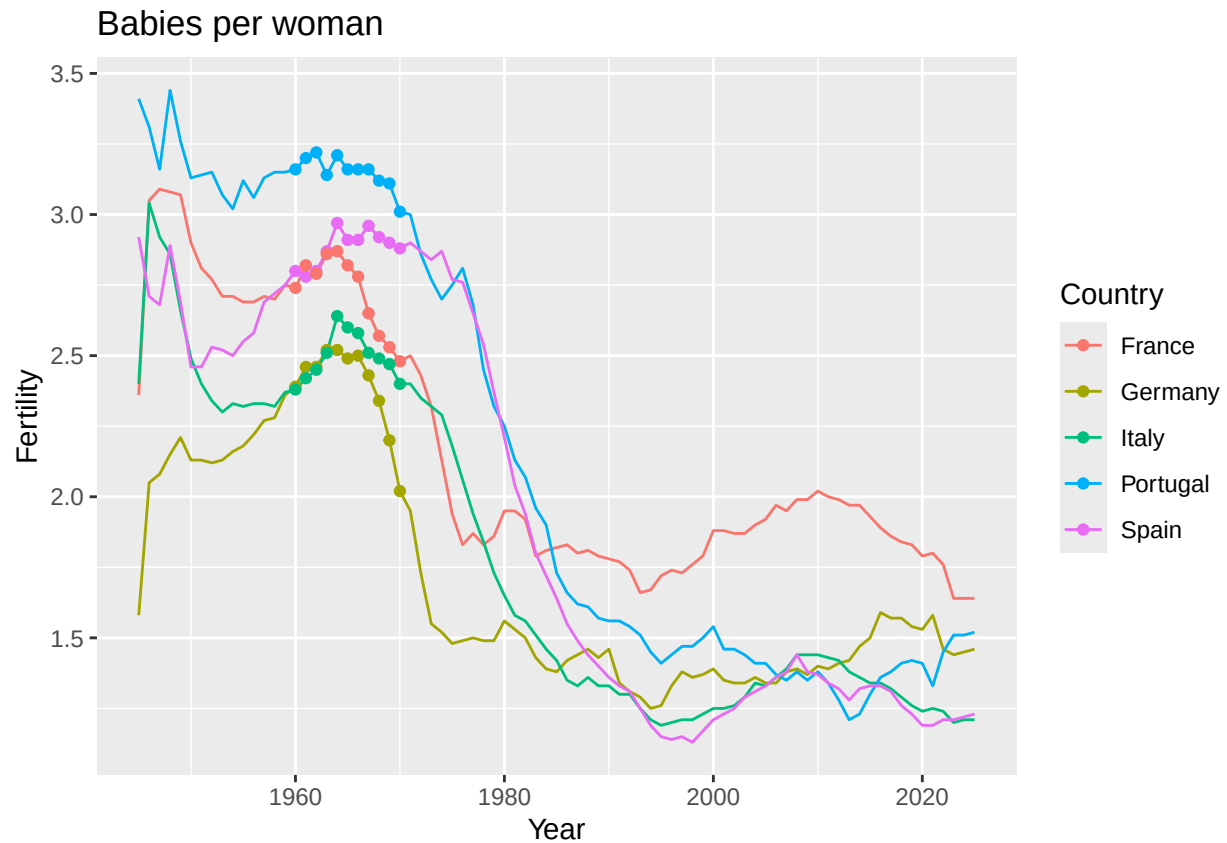
```

Representacion

```

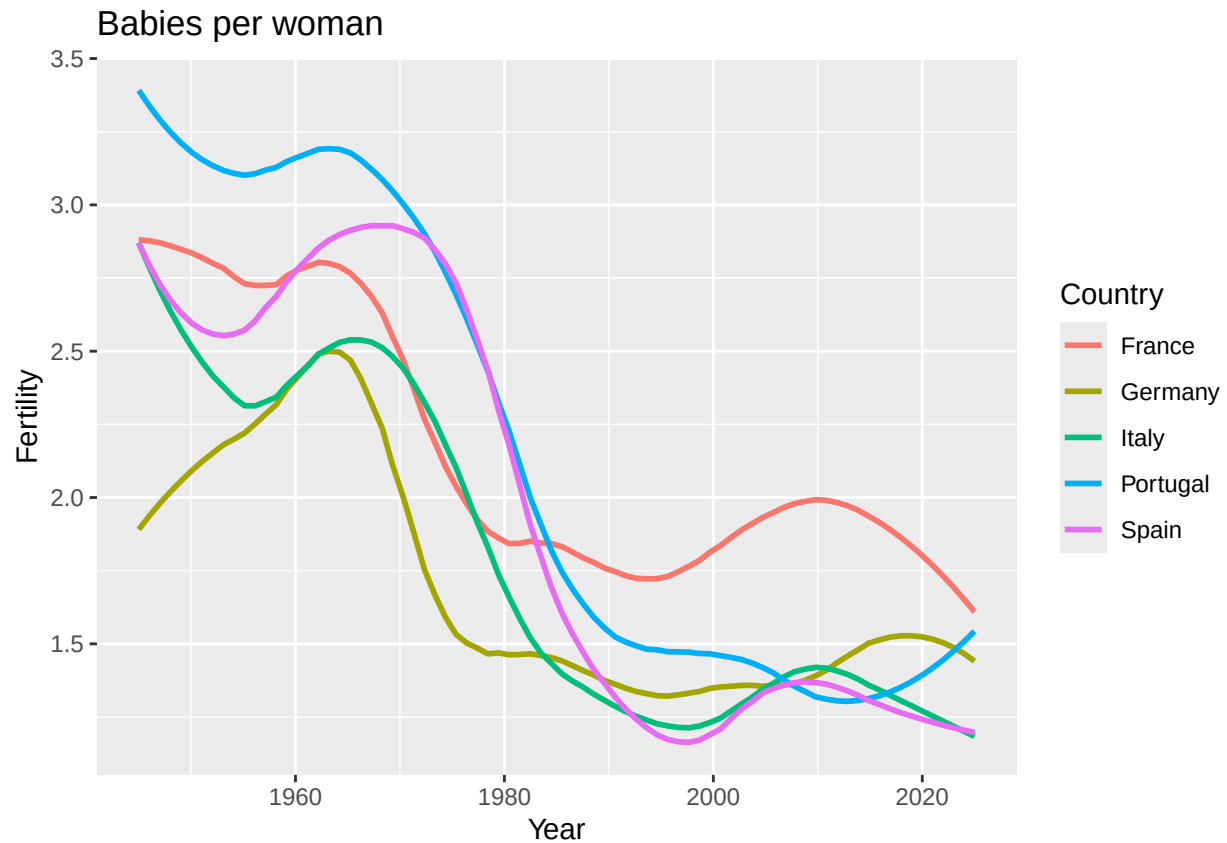
# Representacion 1
df %>%
  filter(Year >= 1945 & Year <= 2025) %>%
  filter(Country %in% c('Spain', 'France', 'Germany', 'Portugal', 'Italy')) %>%
  ggplot(data = ., mapping = aes(x = Year, y = Value, group = Country, color = Country)) +
  geom_line() +
  geom_point(data = ~filter(., Year >= 1960 & Year <= 1970)) + # Para añadir puntos de 1960 a 1980
  labs(title = 'Babies per woman',
       x = 'Year',
       y = 'Fertility')

```



```
# Representacion 2
df %>%
  filter(Year >= 1945 & Year <= 2025) %>%
  filter(Country %in% c('Spain', 'France', 'Germany', 'Portugal', 'Italy')) %>%
  ggplot(data = ., mapping = aes(x = Year, y = Value, group = Country, color = Country)) +
  # geom_line() +
  # geom_point(data = ~filter(., Year >= 1960 & Year <= 1970)) + # Para añadir puntos de 1960 a 1980
  labs(title = 'Babies per woman',
        x = 'Year',
        y = 'Fertility') +
  geom_smooth(se = FALSE, span = 0.3) # Línea suavizada
```

```
## 'geom_smooth()' using method = 'loess' and formula = 'y ~ x'
```



```
# se, oculta la banda de error
# span, controla el suavizado. 0.3 mas suave, 1 mas rigido
```

Ejercicio Tema 6. Textos.

Expresion regular que permita identificar los correos de alumnos de la uv y personal laboral.

```
correos <- c('paco5@alumni.uv.es',
             'vifranjo@uv.es',
             'martsobm@uv.es',
             'pascal@gmail.com',
             'ménfor@alumni.es')

# [] estos corchetes indican una secuencia por caracter.
# Ej: [alumni.], interpreta que tenga a, l, u, m, n, i, o '.'.
# Para evaluar toda la expresion, se debe usar ().
str_match(string = correos, pattern = '^([a-zA-Z0-9]+)@([alumni.]+[uv\\.\\.es])')

##      [,1]
## [1,] "paco5@alumni.uv"
## [2,] "vifranjo@uv"
## [3,] "martsobm@uv"
## [4,] NA
## [5,] NA
```

```
# Expresion correcta
# ^[a-zA-Z0-9]+ = empezar con caracteres de la a-z, A-Z o 0-9, y se puede repetir mas de una vez
# @ = Obligatorio para el correo
# (alumni\\.)? = evalua la expresion 'alumni.', que puede estar o no (?)
# uv\\.es = evalua la expresion 'uv.es', aplica a todos los usuarios
exp_filtro <- '^[a-zA-Z0-9]+@(alumni.)?uv.es'
```

Funciones basicas para manejo de caracteres en R

```
str_detect(string = correos, pattern = exp_filtro)
```

```
## [1] TRUE TRUE TRUE FALSE FALSE
```

```
str_match(string = correos, pattern = exp_filtro)
```

```
##      [,1]      [,2]
## [1,] "paco5@alumni.uv.es" "alumni."
## [2,] "vifranjo@uv.es"    NA
## [3,] "martsobm@uv.es"    NA
## [4,] NA                 NA
## [5,] NA                 NA
```

```
str_match_all(string = correos, pattern = exp_filtro)
```

```
## [[1]]
##      [,1]      [,2]
## [1,] "paco5@alumni.uv.es" "alumni."
##
## [[2]]
##      [,1]      [,2]
## [1,] "vifranjo@uv.es" NA
##
## [[3]]
##      [,1]      [,2]
## [1,] "martsobm@uv.es" NA
##
## [[4]]
##      [,1] [,2]
##
## [[5]]
##      [,1] [,2]
```

```
str_extract(string = correos, pattern = exp_filtro)
```

```
## [1] "paco5@alumni.uv.es" "vifranjo@uv.es"    "martsobm@uv.es"
## [4] NA                 NA
```

```
str_extract_all(string = correos, pattern = exp_filtro)
```

```
## [[1]]  
## [1] "paco5@alumni.uv.es"  
##  
## [[2]]  
## [1] "vifranjo@uv.es"  
##  
## [[3]]  
## [1] "martsobm@uv.es"  
##  
## [[4]]  
## character(0)  
##  
## [[5]]  
## character(0)
```

```
str_which(string = correos, pattern = exp_filtro)
```

```
## [1] 1 2 3
```

```
# En R, gsub() se usa para sustituir globalmente un patrón  
# dentro de una cadena de texto por otro patrón, reemplazando  
# todas las ocurrencias encontradas
```

```
# NOTA: Aca se evalua caracter por caracter. No se comparan/cuentan cada correo del vector  
# sub reemplaza solo la primera
```

```
sub(pattern = '(alumni.)?uv.es|gmail.com', replacement = 'valencia', x = correos)
```

```
## [1] "paco5@valencia" "vifranjo@valencia" "martsobm@valencia"  
## [4] "pascal@valencia" "ménfor@alumni.es"
```

```
# gsub reemplaza todas
```

```
gsub(pattern = '(alumni.)?uv.es|gmail.com', replacement = 'valencia', x = correos)
```

```
## [1] "paco5@valencia" "vifranjo@valencia" "martsobm@valencia"  
## [4] "pascal@valencia" "ménfor@alumni.es"
```

```
# Otro ejemplo para sub y gsub
```

```
texto_prueba <- 'Esto es un texto de prueba para expresiones regulares'
```

```
# Quiero que la letra 'o', sea reemplazada por '5'.
```

```
# sub solo reemplaza el primer elemento
```

```
sub(pattern = 'o', replacement = '5', x = texto_prueba)
```

```
## [1] "Est5 es un texto de prueba para expresiones regulares"
```

```
# gsub lo hace a todos los patrones
gsub(pattern = 'o', replacement = '5', x = texto_prueba)
```

```
## [1] "Est5 es un text5 de prueba para expresi5nes regulares"
```

```
# Devuelve el numero (posicion) en que encuentra el patron
```

```
# Los correos 1, 2 y 3 son los que cumplen el formato de
# @alumni.uv.es o uv.es
grep(pattern = exp_filtro, x = correos)
```

```
## [1] 1 2 3
```

```
# Igual que el anterior, pero con valores bool
grepl(pattern = exp_filtro, x = correos)
```

```
## [1] TRUE TRUE TRUE FALSE FALSE
```

```
texto_prueba
```

```
## [1] "Esto es un texto de prueba para expresiones regulares"
```

```
# strsplit: divide los elementos de una cadena
```

```
# Cada que encuentra una 'a', divide el texto
strsplit(x = texto_prueba, split = 'a')
```

```
## [[1]]
## [1] "Esto es un texto de prueb" " p"
## [3] "r" " expresiones regul"
## [5] "res"
```

```
# Ahora por espacios
```

```
strsplit(x = texto_prueba, split = ' ')
```

```
## [[1]]
## [1] "Esto" "es" "un" "texto" "de"
## [6] "prueba" "para" "expresiones" "regulares"
```

```
# ?str_view_all(string, RegExp)
```

Otro ejercicio

Vocales acentuadas o no en un string

```
texto_vocales <- 'Esto es una expresión. Qué? Sí, contiene varias vocales cón n tildés'
```

```
exp_tildes <- '[aeiouáéíóúAEIOUÁÉÍÓÚ]' # = [v1 | v2 | v3 | v4]... vocal 1 o vocal 2 o vocal 3...
```

```
str_detect(string = texto_vocales, pattern = exp_tildes)
str_match_all(string = texto_vocales, pattern = exp_tildes)
str_extract_all(string = texto_vocales, pattern = exp_tildes)
gsub(pattern = exp_tildes, replacement = 'V', x = texto_vocales)
```

Analisis de texto

```
texto <- c('Con diez cañones por banda,',
          'viento en popa, a toda vela,',
          'no corta el mar, sino vuela,',
          'un velero bergantín.',
          'Bajel pirata que llaman,',
          'por su bravura, El Temido,',
          'en todo mar conocido,',
          'del uno al otro confín.')
```

texto

```
## [1] "Con diez cañones por banda," "viento en popa, a toda vela,"
## [3] "no corta el mar, sino vuela," "un velero bergantín."
## [5] "Bajel pirata que llaman,"      "por su bravura, El Temido,"
## [7] "en todo mar conocido,"        "del uno al otro confín."
```

```
# Evalua las 'lineas' que comienzan por una letra de la a-z, en minúscula.
str_extract_all(string = texto, pattern = '^[a-z]')
str_match_all(string = texto, pattern = '^[a-z]')
```

```
# Evalua las 'lineas' que terminan por una letra de la a-z, en minúscula.
# TODAS terminan en punto o coma.
str_extract_all(string = texto, pattern = '[a-z]$')
str_match_all(string = texto, pattern = '[a-z]$')
```

```
# Evalua las 'lineas' que NO comienzan por una letra de la a-z, en minúscula.
# Las lineas 1 y 5 empiezan por mayúscula
str_extract_all(string = texto, pattern = '^[^a-z]')
str_match_all(string = texto, pattern = '^[^a-z]')
```

```
# Evalua las 'lineas' que NO comienzan por una letra de la a-z, en minúscula.
# Evalua las 'lineas' que comienzan por un caracter diferente a-z
str_extract_all(string = texto, pattern = '^[^a-z]')
str_match_all(string = texto, pattern = '^[^a-z]')
```